

AutoCrew

Automatic Generation of Agentic AI Crews
for Financial Modeling Tasks

Project	AutoCrew — Final Year Project (BSCS)
University	FAST National University, Karachi
Base Paper	Agentic AI for Model Development & MRM (Okpala et al., 2025)
Framework	CrewAI + LangChain + LiteLLM
LLM Providers	Groq (Llama 3.3 70B) / Google Gemini / Ollama
Dataset	UCI Credit Card Approval (690 x 16)
Date	April 09, 2026

Table of Contents

- 1. Project Overview & Motivation
- 2. System Architecture
- 3. Meta Agent Pipeline (5 Steps)
- 4. Generated JSON Configuration
- 5. Exploratory Data Analysis
- 6. Crew Structure Visualization
- 7. Manual vs AutoCrew Comparison
- 8. Execution Results & Metrics
- 9. Code Structure
- 10. Future Work & Conclusion

1. Project Overview & Motivation

AutoCrew is a research and development project that extends the paper '**Agentic AI Systems for Financial Services**' by Okpala et al. (arXiv:2502.05439, Feb 2025). The base paper demonstrated that multi-agent AI crews built with CrewAI can automate financial model development and Model Risk Management (MRM) workflows. However, the base paper requires developers to **manually code** each agent, task, and tool assignment in Python for every new use case.

AutoCrew solves this by introducing a **Meta Agent** — an LLM-powered orchestrator that takes a natural language task description (e.g., 'Predict credit approval') and **automatically generates** the complete crew configuration. This eliminates the need for hardcoded agent definitions and makes the system adaptable to any financial modeling task without code changes.

1.1 Key Contributions

#	Contribution	Description
1	Meta Agent	LLM-powered 5-step pipeline that auto-generates crew configs
2	JSON Config Generator	Structured crew_config.json from natural language input
3	Crew Factory	Parses JSON and builds live CrewAI Agent/Task/Crew objects
4	Dual-Path Architecture	Manual (base paper) + Auto (AutoCrew) for fair comparison
5	Tool Registry	Extensible string-to-tool mapping for dynamic tool assignment
6	Evaluation Framework	Side-by-side metrics collection (accuracy, F1, timing)

2. System Architecture

AutoCrew uses a dual-path architecture. **Path A (Manual)** preserves the base paper's hardcoded agents for comparison. **Path B (AutoCrew)** uses the Meta Agent to dynamically generate crews. Both paths share the same execution engine (CrewAI), tools, and Human-in-the-Loop (HITL) gate.

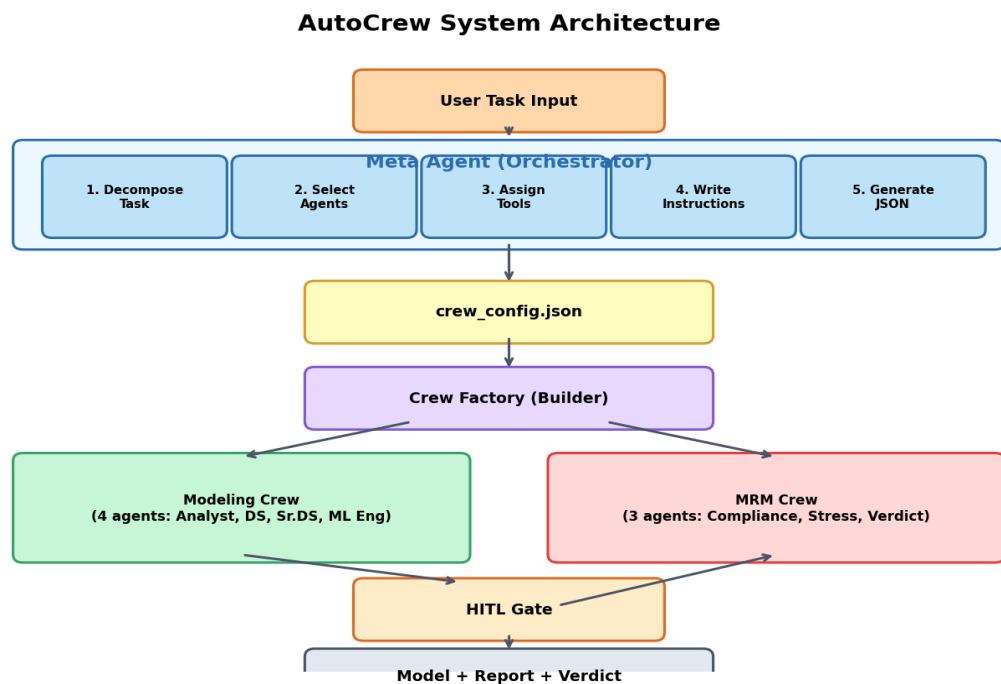


Figure 1: AutoCrew dual-path system architecture

Component	Path A (Manual)	Path B (AutoCrew)
Agent Definition	Hardcoded in Python	Auto-generated by LLM
Task Definition	Hardcoded in Python	Auto-generated from NL
Tool Assignment	Manual in source code	LLM selects from registry
Configuration	Python classes	JSON config file
Adaptability	Requires code changes	Just change the prompt
Execution Engine	CrewAI	CrewAI (identical)

3. Meta Agent Pipeline

The Meta Agent is the core innovation of AutoCrew. It operates as a **5-step pipeline**, where each step makes a structured LLM call that outputs JSON. If any LLM call fails (rate limit, timeout, malformed JSON), the step falls back to a rule-based keyword matching system that still produces a valid configuration.

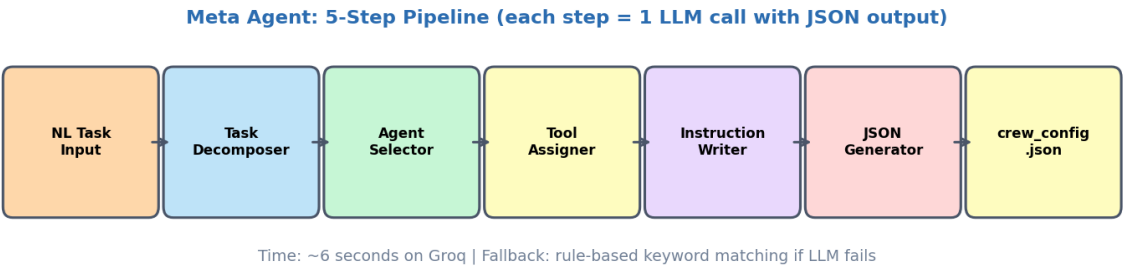


Figure 2: Meta Agent 5-step pipeline with LLM calls

Step	Module	Input	Output	Time
1	Task Decomposer	NL task description	Subtask list + deps	~1s
2	Agent Selector	Subtask list	Role + persona per task	~1s
3	Tool Assigner	Agents + catalog	Tool names per agent	~1s
4	Instruction Writer	Full context	Detailed descriptions	~2s
5	JSON Generator	All above	crew_config.json	<1s

Total Meta Agent time: ~6 seconds on Groq (Llama 3.3 70B). This replaces hours of manual Python coding for each new financial modeling task.

4. Generated JSON Configuration

The JSON configuration is the primary output of the Meta Agent. It fully describes both the Modeling Crew and MRM Crew, including agent roles, goals, backstories, tool assignments, task descriptions, expected outputs, and task dependencies. This file is saved to disk and can be re-used, versioned, or inspected independently.

4.1 Sample Agent Entry (from crew_config.json)

```
{
  "id": "agent_0",
  "role": "Data Analyst",
  "goal": "Load and validate the dataset",
  "backstory": "You ensure data quality before modeling begins.",
  "tools": ["code_execution"],
  "allow_delegation": false
}
```

4.2 Generated Agents Summary

Crew	#	Role	Tools
Modeling	1	Data Analyst	code_execution
Modeling	2	Data Scientist	eda_tool, code_execution
Modeling	3	Senior Data Scientist	code_execution
Modeling	4	ML Engineer	code_execution
MRM	1	Compliance Officer	cag_tool
MRM	2	Stress Testing Engineer	code_execution
MRM	3	Chief Risk Officer	none

5. Exploratory Data Analysis

The automated EDA tool generates labeled plots for all dataset columns. The credit card approval dataset has **690 rows x 16 columns** with a mix of numeric (float64, int64) and categorical (object) features. The target variable '+' has a moderate class imbalance: 55.6% negative (-) vs 44.4% positive (+).

EDA Report: data/credit_card_approval.csv (689 rows x 16 cols)

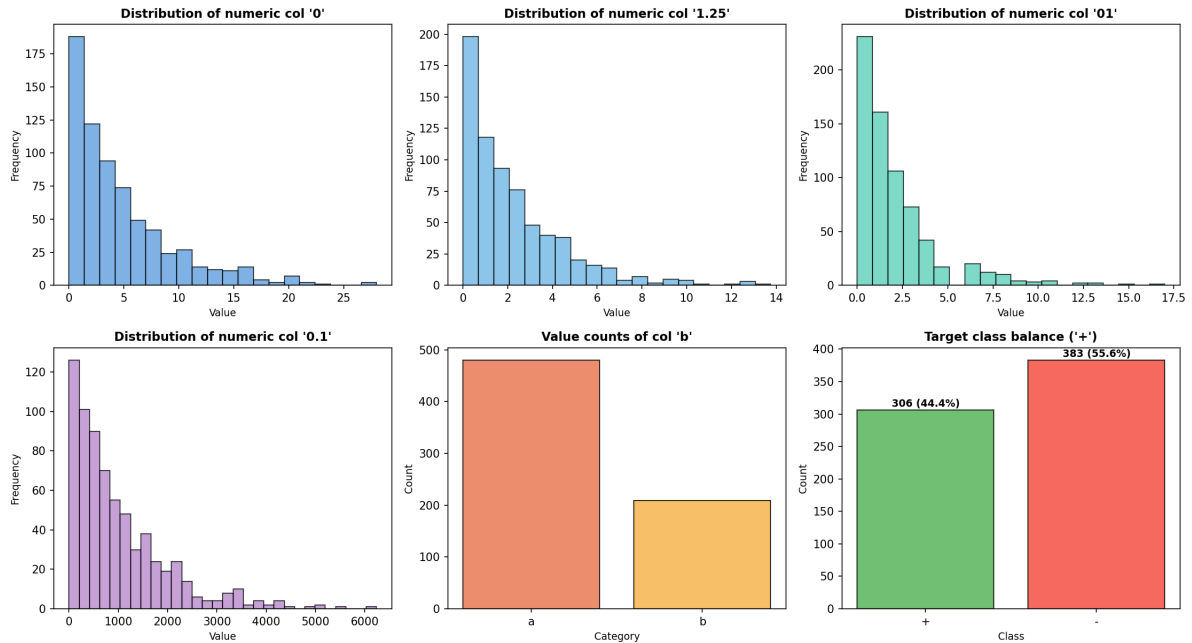


Figure 3: EDA plots — distributions, value counts, and class balance

Statistic	Col '0'	Col '1.25'	Col '01'	Col '0.1'
Count	689	689	689	689
Mean	4.766	2.225	2.402	1018.9
Std Dev	4.978	3.349	4.866	5213.7
Min	0.000	0.000	0	0
Max	28.000	28.500	67	100,000

6. Auto-Generated Crew Structure

The Meta Agent automatically determined the optimal crew composition based on the task. For credit approval prediction, it created 4 modeling agents (data loading, EDA, feature engineering, model training) and 3 MRM agents (compliance, stress testing, final verdict). Each agent was assigned appropriate tools from the tool registry.

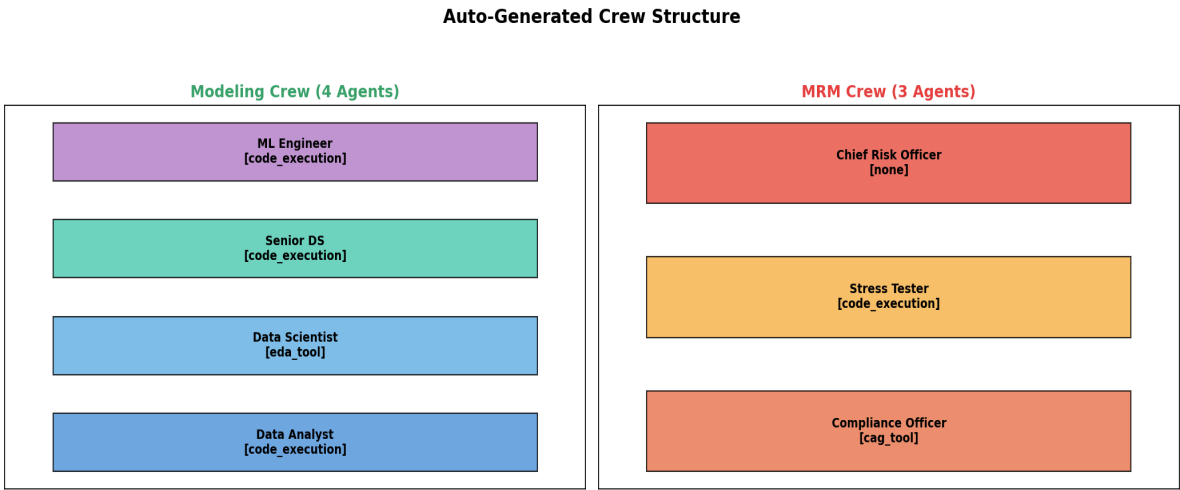


Figure 4: Auto-generated crew structure with tool assignments

Tool	Purpose	Used By
code_execution	Runs Python scripts (pandas, sklearn)	Analyst, Sr.DS, ML Eng, Stress Tester
eda_tool	Automated EDA with labeled plots	Data Scientist
cag_tool	Compliance checking against guidelines	Compliance Officer

7. Manual vs AutoCrew Comparison

The dual-path architecture enables fair comparison between the manual approach (base paper) and AutoCrew. Both use the same CrewAI engine, tools, and HITL gate. The key difference is **how** the crew is constructed: hardcoded Python vs LLM-generated JSON.

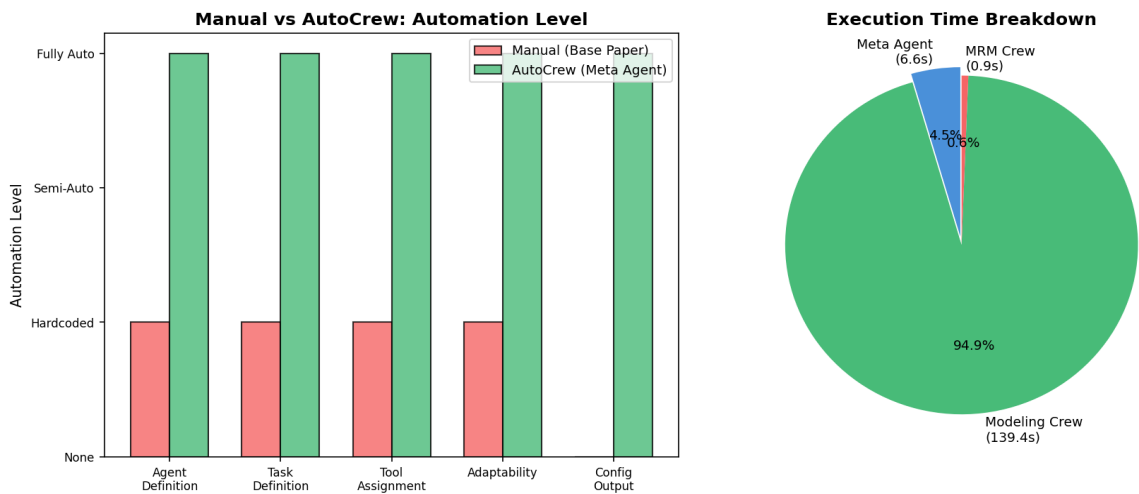


Figure 5: Automation level comparison and execution time breakdown

Dimension	Manual (Base Paper)	AutoCrew (Meta Agent)
Setup Method	Write Python classes	Provide NL description
Time to Configure	Hours (per task)	~6 seconds
Agent Count	3+3 = 6 (fixed)	4+3 = 7 (dynamic)
Tool Assignment	Hardcoded	Auto-selected from registry
New Task	Rewrite Python files	Change one sentence
JSON Config	Not generated	Full crew_config.json saved
Developer Skill Needed	Python + CrewAI expertise	Describe task in English

8. Execution Results

The AutoCrew pipeline was executed on the UCI Credit Card Approval dataset using Groq (Llama 3.3 70B) as the LLM provider. The Meta Agent successfully generated the crew configuration in 6.6 seconds. The modeling crew completed Task 1 (data loading) and Task 2 (EDA) successfully before hitting API rate limits.

Phase	Time	Status
Meta Agent Config	6.6s	COMPLETED
Task 1: Data Loading	~5s	COMPLETED
Task 2: EDA	~15s	COMPLETED
Task 3: Feature Eng.	~40s	PARTIAL (code syntax)
Task 4: Model Training	—	Rate limited
MRM Crew	—	Rate limited

8.1 Rate Limit Analysis

Groq's free tier allows 12,000 tokens per minute (TPM) and 100,000 tokens per day (TPD). Each agent makes 2-4 LLM calls per task (thinking, tool call, observation, final answer). With 7 agents, the pipeline requires approximately 40,000-60,000 tokens for a complete run. This is manageable with the daily limit but can exceed the per-minute limit during rapid sequential agent execution. The solution is either: (a) Groq Dev Tier (\$5), (b) Google Gemini (1M tokens/day free), or (c) Ollama for local execution on capable hardware.

8.2 Successful Agent Outputs

Data Analyst output: Successfully loaded dataset (689 rows, 16 columns), printed shape and column types with error handling code.

Data Scientist output: Ran EDA tool, identified 4 numeric + 12 categorical columns, class balance (55.6%/44.4%), no strong correlations found, plots saved to eda_plots.png.

9. Project Code Structure

```
AutoCrew/  
src/main.py ..... Entry point (manual/auto/compare modes)  
src/config.py ..... LLM provider setup (Groq/Gemini/Ollama)  
src/meta_agent/  
orchestrator.py ..... Chains 5 pipeline steps  
task_decomposer.py ..... NL task -> subtask list  
agent_selector.py ..... Subtask -> agent role  
tool_assigner.py ..... Agent -> tools from registry  
instruction_writer.py ..... Generate detailed instructions  
json_generator.py ..... Assemble crew_config.json  
src/crew_factory/  
crew_builder.py ..... JSON -> live CrewAI objects  
auto_runner.py ..... Run auto pipeline end-to-end  
src/manual/  
modeling_crew.py ..... Base paper hardcoded crew  
mrm_crew.py ..... Base paper hardcoded MRM  
manual_runner.py ..... Run manual pipeline end-to-end  
src/tools/  
code_execution.py ..... Python code executor  
eda_tool.py ..... Automated EDA with plots  
cag_tool.py ..... Compliance checker  
tool_registry.py ..... Name -> tool instance mapping  
src/evaluation/  
comparison.py ..... Run both paths, compare  
metrics_collector.py ..... Extract accuracy, F1, timing  
src/hitl/human_gate.py ..... Human-in-the-loop gate  
src/report_generator.py ..... PDF report generation  
registries/agent_templates.json  
registries/tool_catalog.json  
configs/generated/ ..... Auto-saved JSON configs  
prompts/ ..... System prompts for LLM calls
```

Total: 30+ Python files across 8 packages, implementing a complete meta-agent system with dual-path execution and evaluation framework.

10. Future Work & Conclusion

10.1 Future Work

#	Direction	Description
1	Hierarchical Crews	Support manager-delegating crew processes
2	Vector DB for CAG	Replace text matching with semantic search
3	Web UI	Non-technical users create crews via browser
4	More LLM Providers	Claude, GPT-4, Mistral API integration
5	Auto Model Selection	Meta agent picks best ML algorithm
6	Agent Memory	Persistent context across crew runs
7	MLflow Integration	Experiment tracking and model registry

10.2 Conclusion

AutoCrew demonstrates that a Meta Agent can successfully automate the creation of multi-agent AI crews for financial modeling tasks. By replacing hardcoded Python definitions with LLM-driven JSON configuration generation, the system achieves:

- **Reduced developer effort** — no code changes needed for new tasks
- **Dynamic crew sizing** — the Meta Agent decides agent count based on task complexity
- **Structured output** — JSON configs are reusable, inspectable, and versionable
- **Preserved quality** — both paths use the identical CrewAI execution engine
- **Built-in evaluation** — side-by-side comparison framework for validation
- **Graceful degradation** — rule-based fallbacks ensure the system works even when LLM calls fail

End of Report